

CUDA Readiness Checkpoint

The bar for leaving Phase 0.

Closed book. No notes, no compiler, no searching, except where a part explicitly says to write and run code. Answers are not provided anywhere in this repository -- that is deliberate. A checkpoint with an answer key tests reading comprehension.

The point

This does **not** test whether C++ is finished. Nobody finishes C++.

It tests whether enough foundation exists to start CUDA and keep learning C++ alongside it. Passing means starting Phases 1-3 immediately. Failing means specific modules to revisit -- the parts are organized so the gaps are obvious.

PART A — Theory

Answer in your own words, in full sentences.

1. What does the compiler know about a variable that a Python interpreter does not?
 2. Why is `sizeof(int)` free at run time?
 3. What are the four stages of the build, and what does each do?
 4. How do you distinguish a compile error, a linker error, and a run-time error?
 5. What is a stack frame? What happens to it when a function returns?
 6. Why can arrays not be passed by value?
 7. What is array decay, and what information is lost?
 8. Given `int x = 10; int* p = &x;` -- explain `x`, `&x`, `p`, `*p`, `&p`.
 9. Why does `p + 1` advance by `sizeof(T)` rather than by one byte?
 10. Why is `a[i]` identical to `*(a + i)`?
 11. What are the three ways a pointer becomes invalid? Which is most dangerous?
 12. Stack versus heap: allocation cost, lifetime, size limit, and when to use each.
 13. What is a memory leak? Why can that memory not be recovered?
 14. What is a cache line, and why does it make sequential access fast?
 15. What does "memory bound" mean, and how would you tell whether an operation is?
 16. Why is `sizeof(struct)` usually larger than the sum of its fields?
 17. What is the difference between AoS and SoA? Which suits reading one field of every element?
 18. Pass by value, by pointer, by reference -- when do you use each?
-

PART B — Predict the output

Write every answer before checking any of them.

```
int a[5] = {10, 20, 30, 40, 50};
int* p = a + 2;
```

1. `*p`
2. `*(p + 1)`
3. `p - a`
4. `p[2]`
5. `*p + 5`

```
int x = 7, y = 2;
```

6. `x / y`
7. `(float)x / y`
8. `x % y`

```
void f(int v) { v = 99; }  
void g(int* v) { *v = 99; }  
int n = 1;
```

9. After `f(n)`, what is `n`?
10. After `g(&n)`, what is `n`?

```
struct S { char a; int b; };  
struct T { int b; char a; };
```

11. `sizeof(S)`
12. `sizeof(T)`

```
unsigned int u = 0;
```

13. `u - 1`

```
float arr[4];
```

14. `sizeof(arr)`
15. Byte gap between `&arr[0]` and `&arr[1]`

PART C — Memory reasoning

Show your working.

1. `float data[16]` begins at address `0x1000`. What is the address of `data[9]`?

2. `double d[8]` begins at 0x2000. What is the address of `d[5]`?
3. A 12x20 matrix, flat and row-major. What is the flat index of element (7, 13)?
4. Same matrix, of floats. What is the byte offset of (7, 13) from the base?
5. How many bytes does an array of 5,000,000 floats occupy? In MB?
6. The same count as doubles?
7. `out[i] = a[i] * b[i]` over n floats. How many bytes are read? Written? Total?
8. A struct with 20 bytes of fields but `sizeof` 24. For 2,000,000 elements, how many bytes are wasted?
9. Particles with 6 floats each. Reading only the x field: what is the stride in AoS? In SoA?
10. A 64-byte cache line, reading floats with stride 8. How many useful bytes per line?
11. `int arr[10000000]` as a local variable. What happens, and why?
12. An operation moving 240 MB on hardware sustaining 300 GB/s. What is the floor on its runtime? Why a floor and not a prediction?

PART D — Debugging

For each: name the bug, state the stage at which it surfaces, and give the fix.

```
// 1.
int sum(int* arr) {
    int n = sizeof(arr) / sizeof(arr[0]);
    int s = 0;
    for (int i = 0; i < n; ++i) s += arr[i];
    return s;
}

// 2.
int* create() {
    int arr[100];
    return arr;
}

// 3.
int a[5];
for (int i = 0; i <= 5; ++i) a[i] = i;

// 4.
int* p = new int[10];
delete p;

// 5.
void process() {
    int* buf = new int[1000];
    if (something()) return;
    delete[] buf;
}

// 6.
int helper(int x);
int main() { return helper(5); }
```

```
// 7.  
for (unsigned int i = n - 1; i >= 0; --i) { }  
  
// 8.  
int x;  
int total = x + 5;  
  
// 9.  
int* p = new int[10];  
delete[] p;  
p[0] = 1;  
  
// 10.  
double result = 7 / 2;
```

PART E — Coding

Write these from an empty file. Compiling and running them is expected.

E1. Implement, from memory:

```
void vector_add(const float* a, const float* b, float* out, int n)
```

Then explain every part of the signature: why `const` on two parameters and not the third, why pointers rather than arrays, why `n` is needed at all.

E2. Implement:

```
void scale(float* data, int n, float factor)
```

in two versions -- one with indexing, one with pointer arithmetic only.

E3. Implement a function computing the mean and standard deviation of a float array, returning both through output parameters. Handle `n == 0`.

E4. Implement a function normalizing an array in place to zero mean and unit standard deviation. State how many passes over the data it makes.

E5. Allocate a rows x cols matrix on the heap, fill element (r,c) with `r * cols + c`, print it as a grid, and free it. No leaks under `-fsanitize=address`.

E6. Define a struct with at least five fields of mixed types, ordered to minimize `sizeof`. State the size and justify the ordering.

PART F — Explain in your own words

Write a paragraph for each, as if explaining to a competent programmer who does not know C++.

F1. Given

```
void scale(float* data, int n, float factor)
```

explain:

- what `data` contains
- where the elements it refers to are stored
- how `data[i]` is computed
- what `data + i` means
- how this function modifies the caller's data
- roughly how many bytes it accesses for `n = 1,000,000`
- why contiguous memory matters here

F2. Why do pointers exist? What problem do they solve that values cannot?

F3. Why does data layout affect performance? Use a concrete example.

F4. Why is `float` preferred over `double` in GPU deep learning work?

F5. What is the difference between a compile-time and a run-time property? Give three examples of each.

Scoring

Part	Questions	Pass mark
A — Theory	18	14
B — Predict	15	12
C — Memory reasoning	12	9
D — Debugging	10	8
E — Coding	6	6
F — Explanation	5	4

Part E must be 6/6. Not most of it -- all of it, written unaided and compiling. Everything in Phase 3 is built out of those six functions.

The real bar

Everything above is diagnostic. The actual criterion is two things:

1. Look at

```
void scale(float* data, int n, float factor)
```

and explain, without hesitating: what `data` contains, where the elements live, how `data[i]` is computed, what `data + i` means, how the function modifies the caller's data, roughly how many bytes it touches, and why contiguity matters.

2. Write

```
vector_add(const float* a, const float* b, float* out, int n)
```

on the CPU, unaided, and explain the memory involved.

If both are true, Phase 0 is done. Start CUDA. Continue C++ in parallel, driven by what the kernel work demands.

If either is not, this phase is not finished -- regardless of how many exercises have been completed. The gaps will be in Modules 03 and 04, and those are the ones to revisit.

After passing

- ☐ Record the result in [learning-log.md](#)
- ☐ Tick the Phase 0 boxes in [README.md](#)
- ☐ Update the **Current focus** block in [../..../ROADMAP.md](#) to Phases 1-3
- ☐ Begin [fundamentals/01-vector-add](#)

The last one is the point of all of this.